

Add images to the Dice Roller app

About this codelab

1. Before you begin

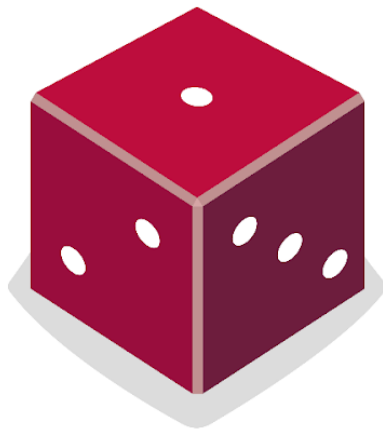
In this codelab, you will add dice images to your existing Dice Roller Android app. Be sure to complete the earlier codelab on building the foundation of the Dice Roller app first.

Instead of displaying the value of the dice roll in a `TextView`, your app will display the appropriate dice image for the number of sides that was rolled. It will be a much more visual and enhanced user experience for your app.

12:50



Dice Roller



ROLL



You will be provided with a link to download the dice images, and you will add them as resources in your app. To write code for which dice image to use, you will be using a `when` statement in Kotlin.

Prerequisites

- Completed the Create an interactive Dice Roller app codelab.
- Able to write control flow statements (`if / else`, `when` statements).
- Able to update the UI of the app based on user input (modifying the `MainActivity.kt` file).
- Able to add a click listener to a `Button`.
- Able to add image resources to an Android app.

What you'll learn

- How to update an `ImageView` while the app is running.
- How to customize your app behavior based on different conditions (using a `when` statement).

What you'll build

- Dice Roller Android app that has a `Button` to roll a dice and update the image on the screen.

What you need

- A computer with Android Studio installed.
- Internet connection to download the dice images.

[2. Update the layout for the app](#)

In this task, you'll replace the `TextView` in your layout with an `ImageView` that displays an image of the dice roll result.

Open Dice Roller app

1. Open and run the Dice Roller app from the previous codelab in Android Studio. You can use the [solution code](#) or the code you created.

The app should look like this.

1:16



Dice Roller

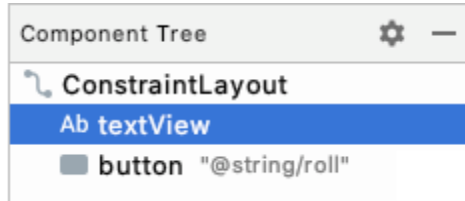
2

ROLL

2. Open `activity_main.xml` (**app > res > layout > activity_main.xml**). This opens the **Layout Editor**.

Delete the TextView

1. In the **Layout Editor**, select the `TextView` in the **Component Tree**.

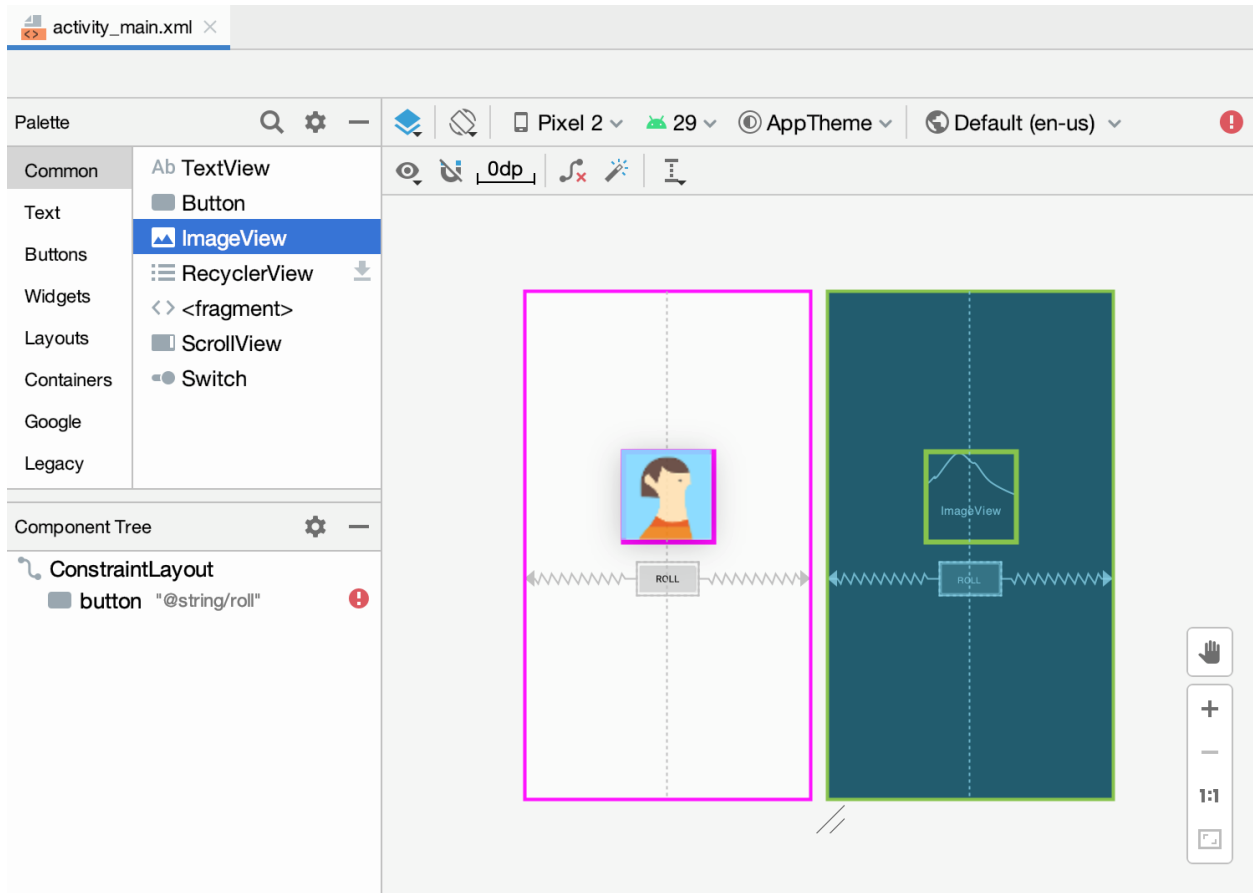


Tip: As you add more UI components and are adding and removing constraints, you may temporarily find one `View` overlapping another, making it hard to select the one in the back. In that case, you can select a `View` by selecting it in the **Component Tree** instead.

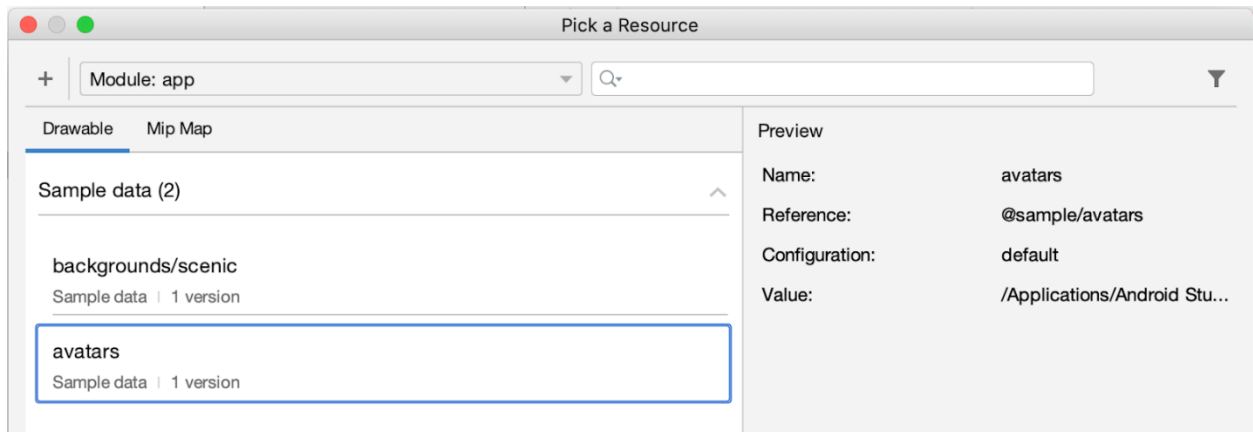
2. Right-click and choose **Delete** or press the `Delete` key.
3. Ignore the warning on the `Button` for now. You'll fix that in the next step.

Add an ImageView to the layout

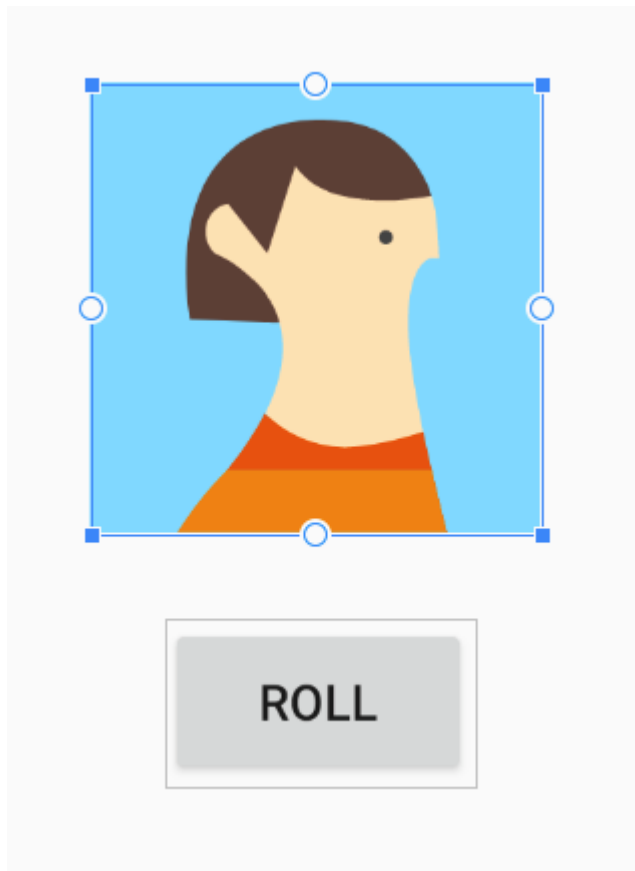
1. Drag an `ImageView` from the **Palette** onto the **Design** view, positioning it above the `Button`.



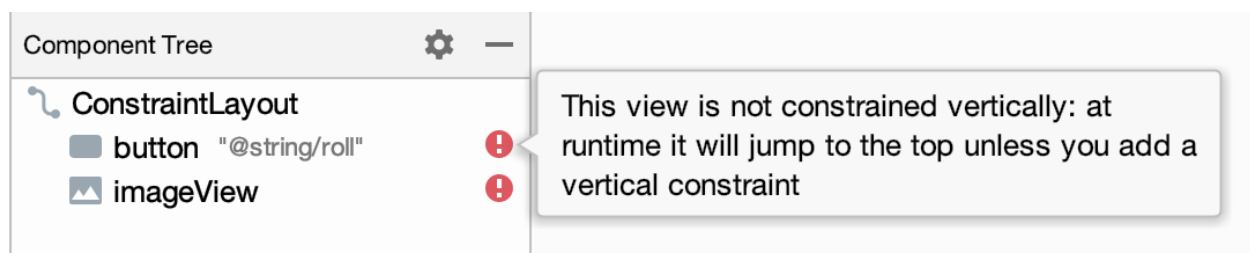
2. In the **Pick a Resource** dialog, select **avatars** under **Sample data**. This is the temporary image you will use until you add the dice images in the next task.



3. Press **OK**. The **Design** view of your app should look like this.



4. In the **Component Tree**, you will notice two errors. The `Button` is not vertically constrained, and the `ImageView` is neither vertically nor horizontally constrained.



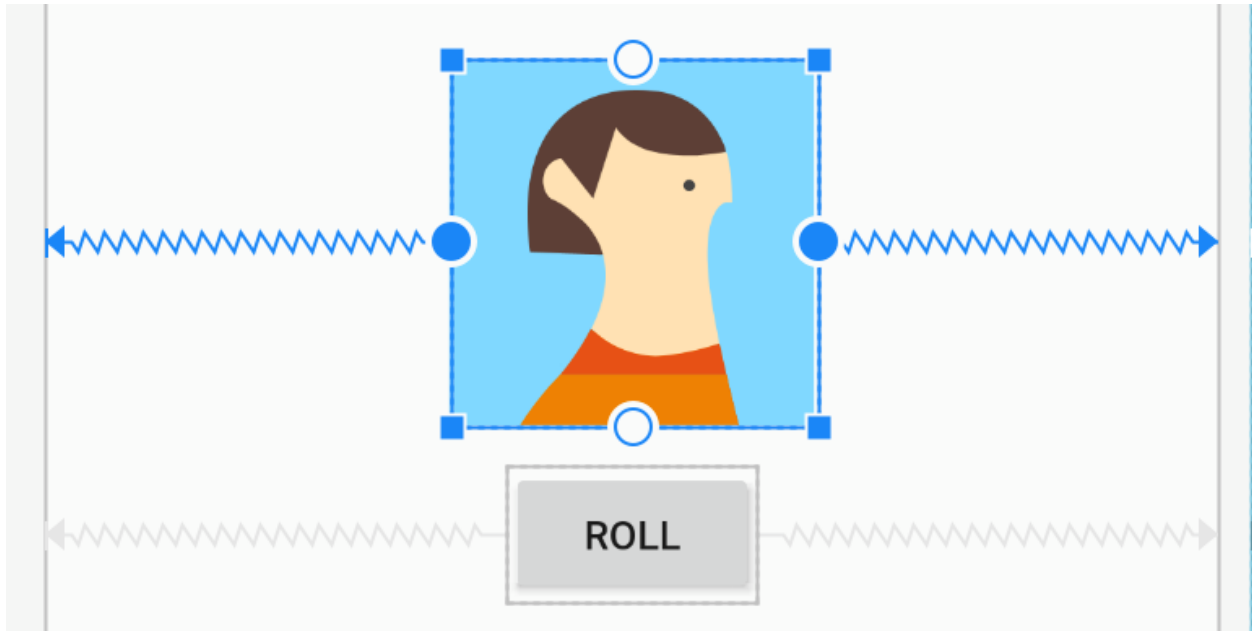
The `Button` is not vertically constrained because you removed the `TextView` below which it was originally positioned. Now you need to position the `ImageView` and the `Button` below it.

Position the `ImageView` and `Button`

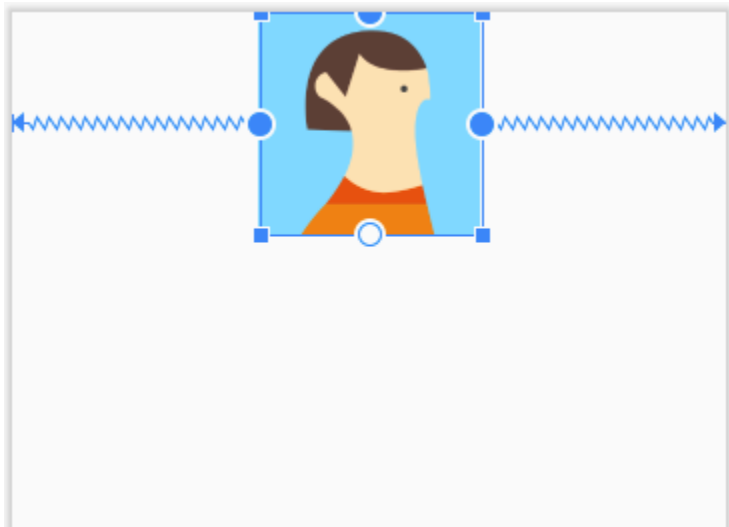
You need to vertically center the `ImageView` in the screen, regardless of where the `Button` is located.

1. Add horizontal constraints to the `ImageView`. Connect the left side of the `ImageView` to the left edge of the parent `ConstraintLayout`.

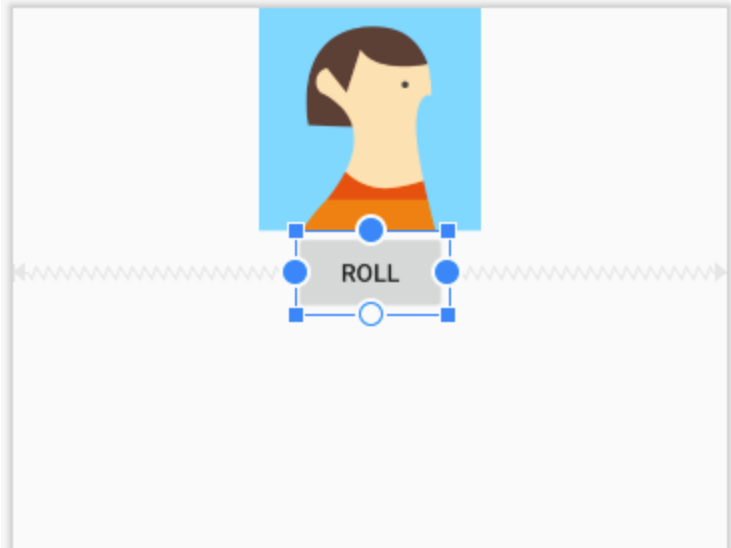
2. Connect the right side of the `ImageView` to the right edge of the parent. This will horizontally center the `ImageView` within the parent.



3. Add a vertical constraint to the `ImageView`, connecting the top of the `ImageView` to the top of the parent. The `ImageView` will slide up to the top of the `ConstraintLayout`.



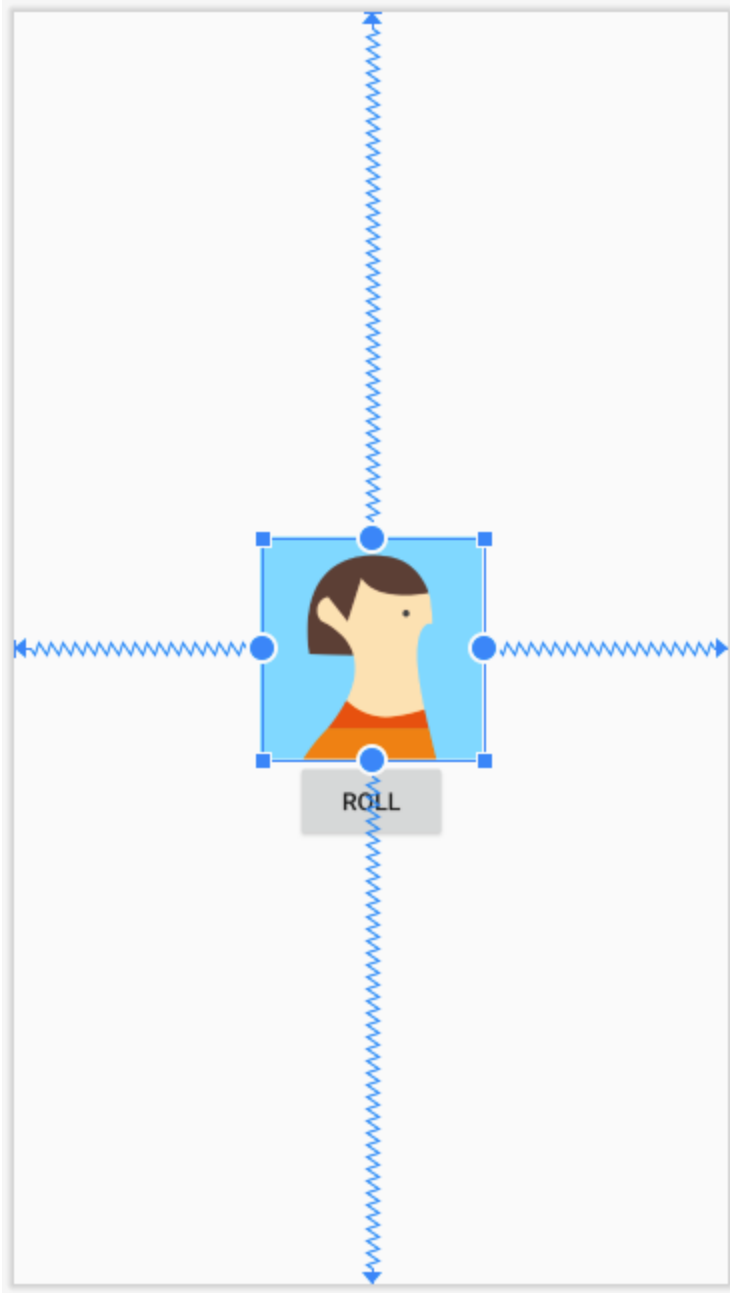
4. Add a vertical constraint to the `Button`, connecting the top of the `Button` to the bottom of the `ImageView`. The `Button` will slide up beneath the `ImageView`.



5. Now select the `ImageView` again and add a vertical constraint connecting the bottom of the `ImageView` to the bottom of the parent. This centers the `ImageView` vertically in the `ConstraintLayout`.

All the warnings about constraints should now be gone.

After all that, the **Design** view should look like this, with the `ImageView` in the center and the `Button` just below it.



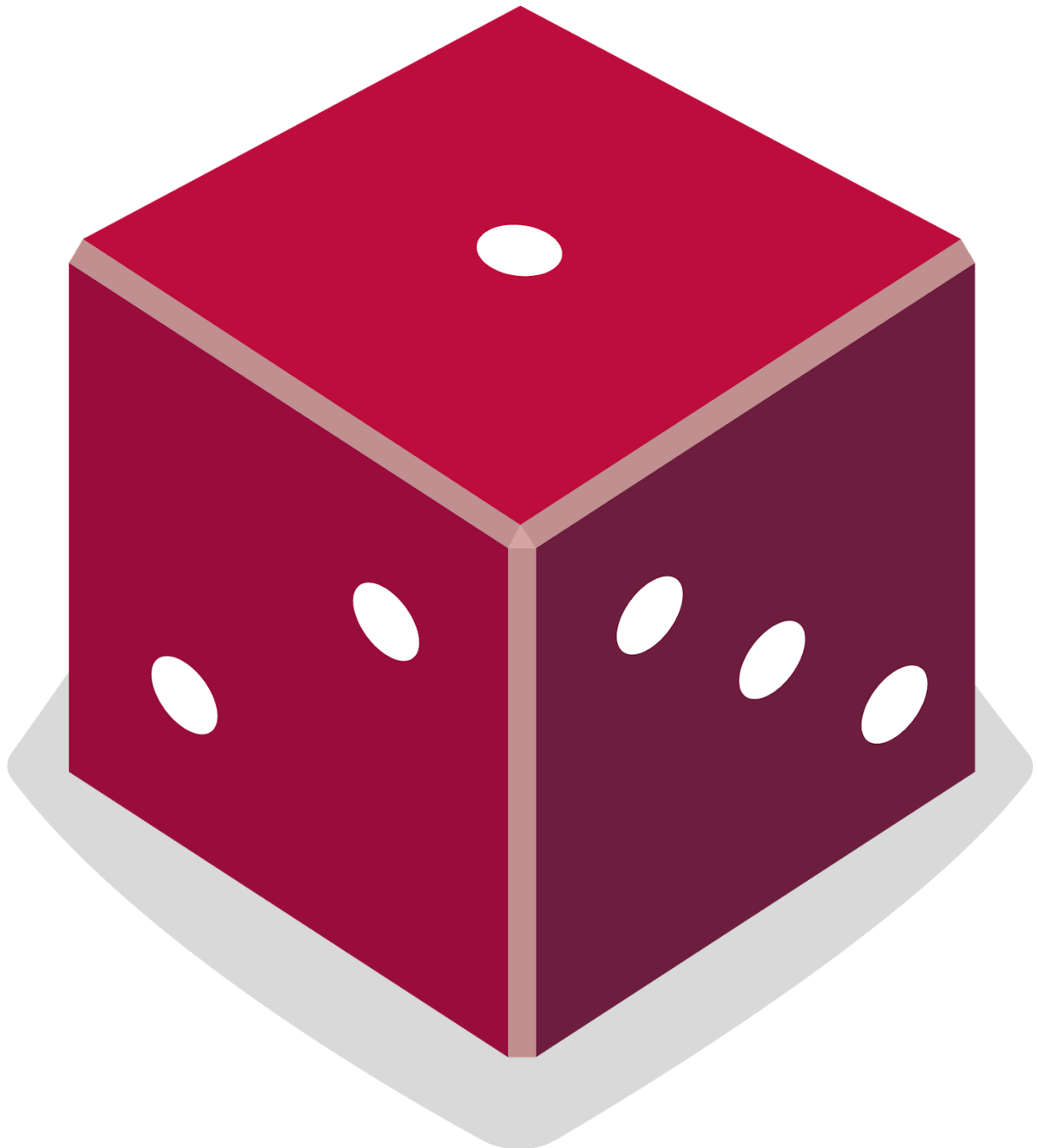
You may notice a warning on the `ImageView` in the **Component Tree** that says to add a content description to your `ImageView`. Don't worry about this warning for now because later in the codelab, you will be setting the content description of the `ImageView` based on what dice image you're displaying. This change will be made in the Kotlin code.

[3. Add the dice images](#)

In this task, you'll download some dice images and add them to your app.

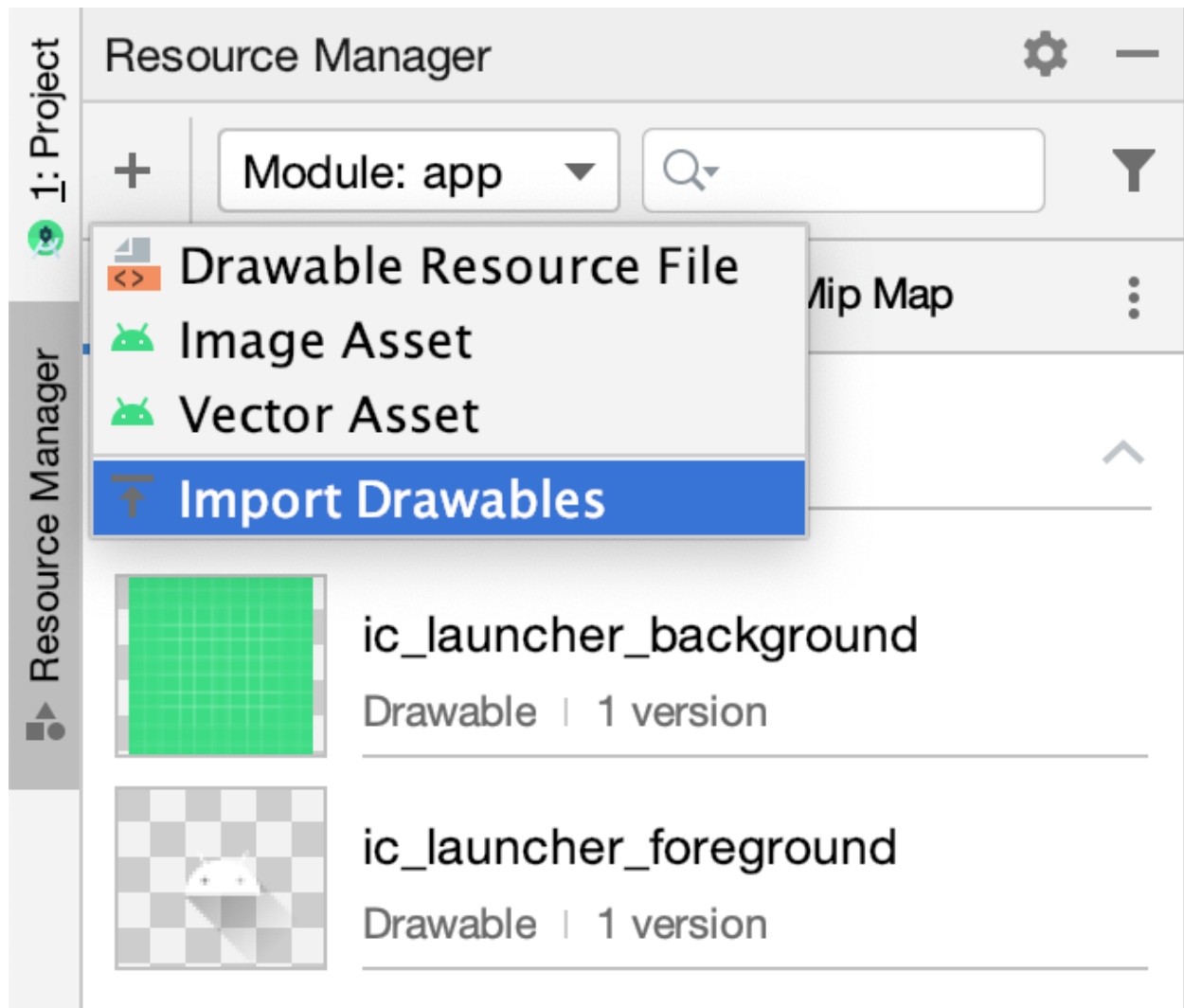
Download the dice images

1. Open [this URL](#) to download a zip file of dice images to your computer. Wait for the download to complete.
2. Locate the file on your computer (likely in the **Downloads** folder).
3. Double-click the zip file to unpack it. This creates a new `dice_images` folder that contains 6 dice image files, displaying the dice values from 1 to 6.

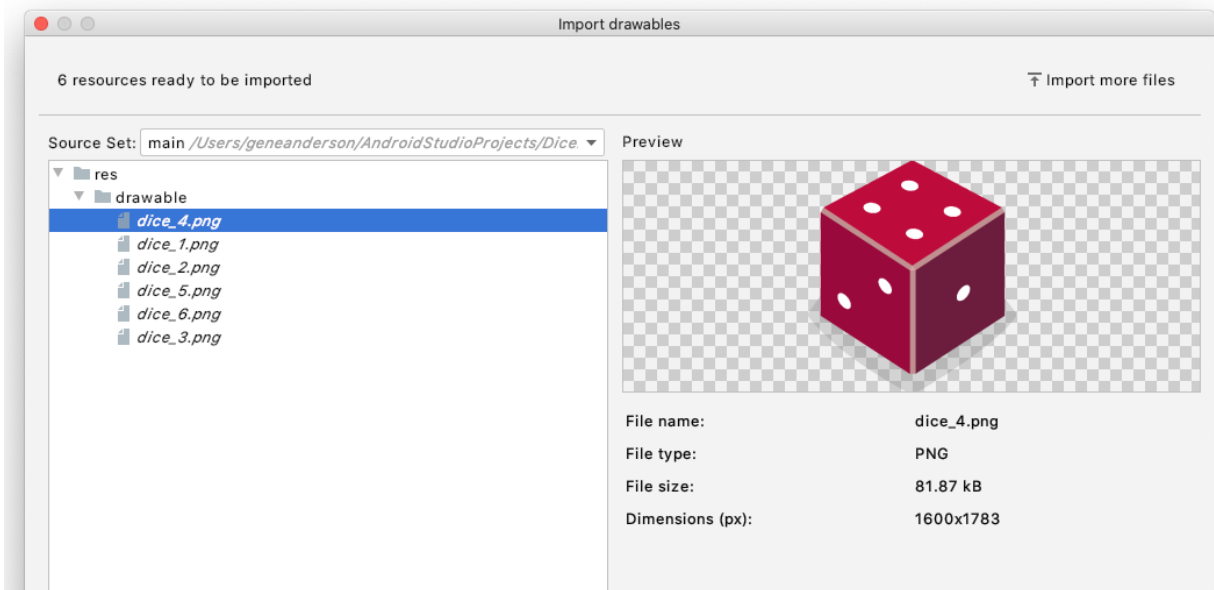
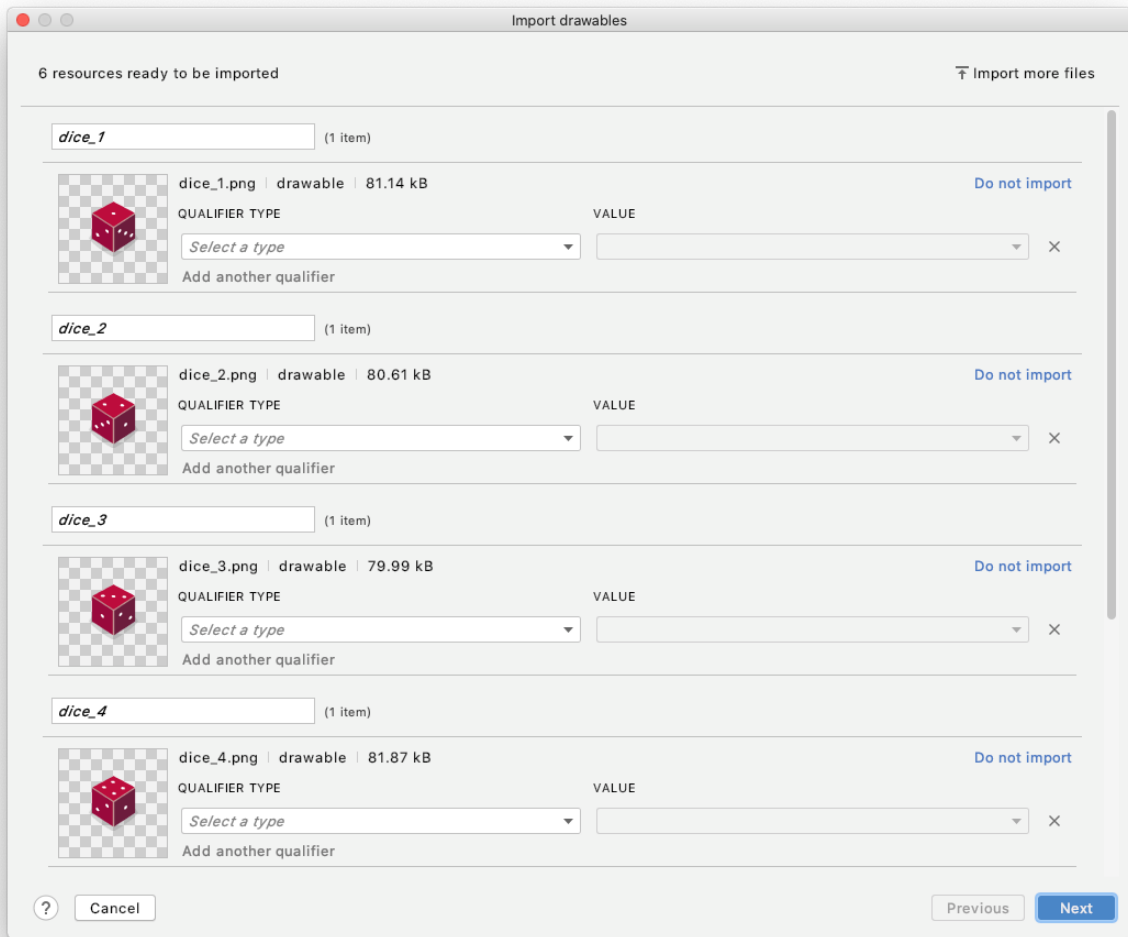


Add dice images to your app

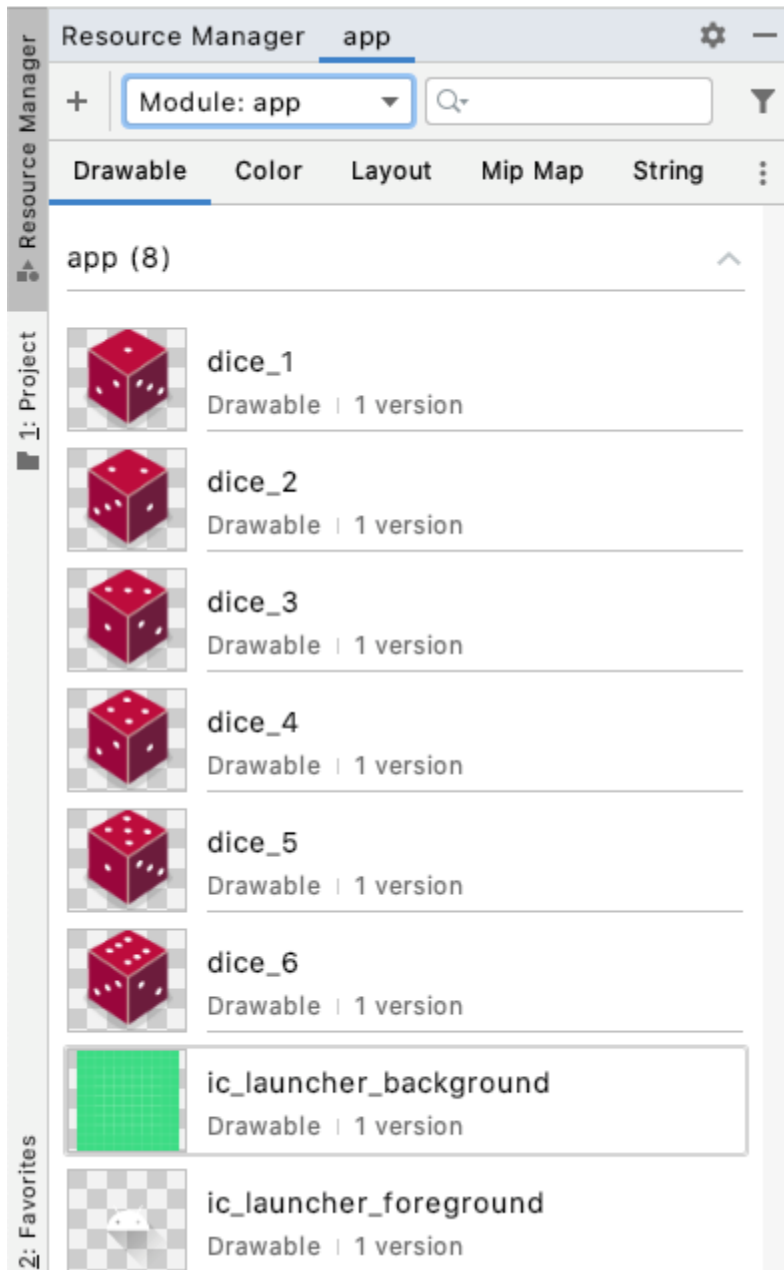
1. In Android Studio, click on **View > Tool Windows > Resource Manager** in the menus or click on the **Resource Manager** tab to the left of the **Project** window.
2. Click the + below **Resource Manager**, and select **Import Drawables**. This opens a file browser.



3. Find and select the 6 dice image files. You can select the first file, then while holding down the **Shift** key, select the other files.
4. Click **Open**.
5. Click **Next** and then **Import** to confirm that you want to import these 6 resources.



6. If the files were imported successfully, the 6 images should appear in the Resource Manager (**app>res>drawable**) for your app.



Nice work! In the next task, you will make use of these images in your app.

Important! - You will be able to refer to these images in your Kotlin code with their resource IDs:

- `R.drawable.dice_1`
- `R.drawable.dice_2`
- `R.drawable.dice_3`

- `R.drawable.dice_4`
- `R.drawable.dice_5`
- `R.drawable.dice_6`

4. Use the dice images

Replace the sample avatar image

1. In the **Design Editor**, select the `ImageView`.
2. In **Attributes** in the **Declared Attributes** section, find the tool **srcCompat** attribute, which is set to the avatar image.

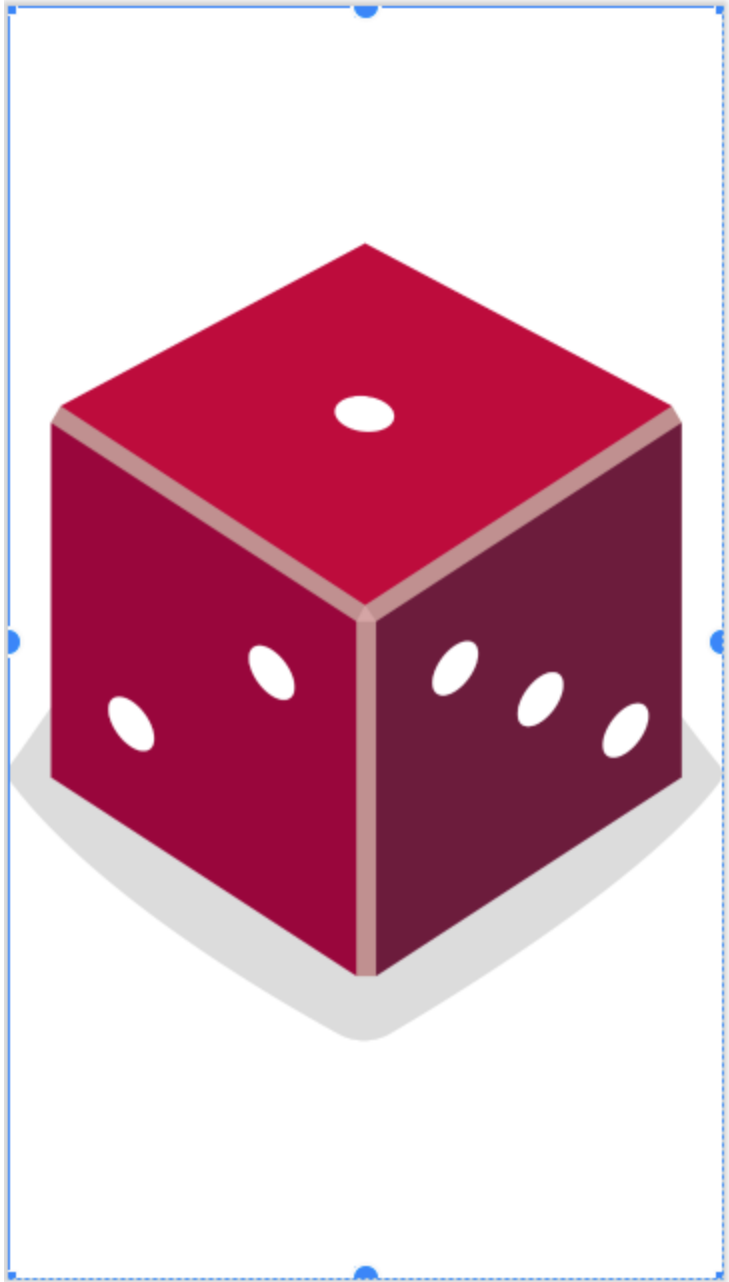
Remember that the tools **srcCompat** attribute uses the provided image only inside the **Design** view of Android Studio. The image is only displayed to developers as you build the app, but will not be seen when you actually run the app on the emulator or on a device.

3. Click the tiny preview of the avatar. This opens a dialog to pick a new resource to use for this `ImageView`.



4. Select the `dice_1` drawable and click **OK**.

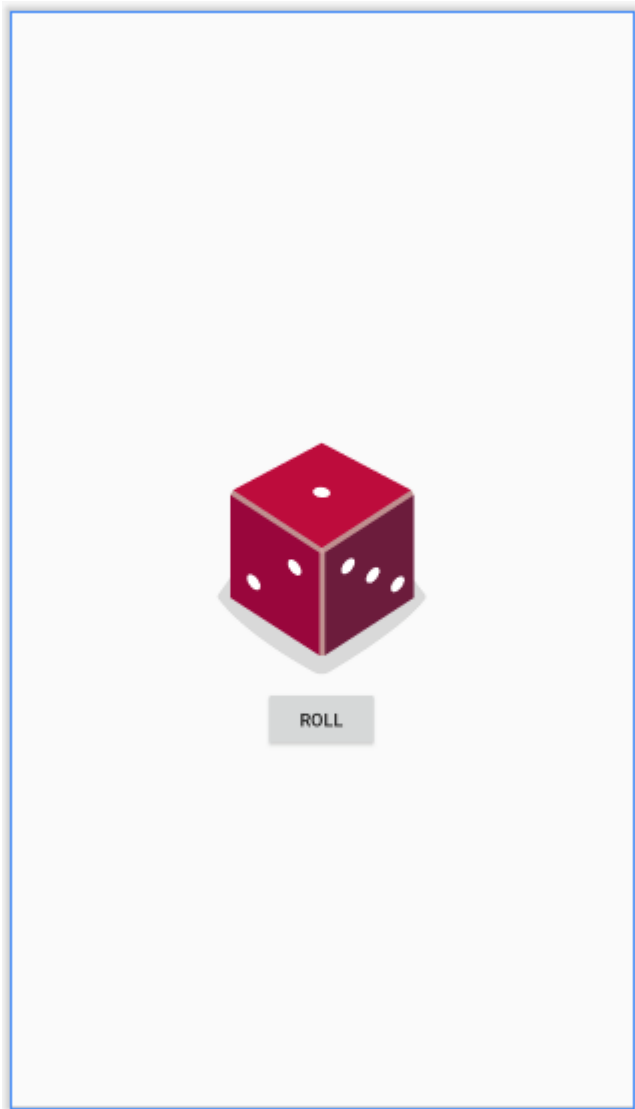
Whoa! The `ImageView` takes up the whole screen.



Next, you'll adjust the width and height of the `ImageView`, so it doesn't hide the `Button`.

5. In the **Attributes** window under the **Constraints Widget**, locate the **layout_width** and **layout_height** attributes. They are currently set to **wrap_content**, meaning that the `ImageView` will be as tall and as wide as the content (the source image) inside it.
6. Instead, set a fixed width of 160dp and fixed height of 200dp on the `ImageView`. Press **Enter**.

The `ImageView` is much smaller now.



You might find the `Button` is a little too close to the image.

7. Add a top margin to the button of 16dp by setting it in the **Constraint Widget**.

id	button
text	@string/roll

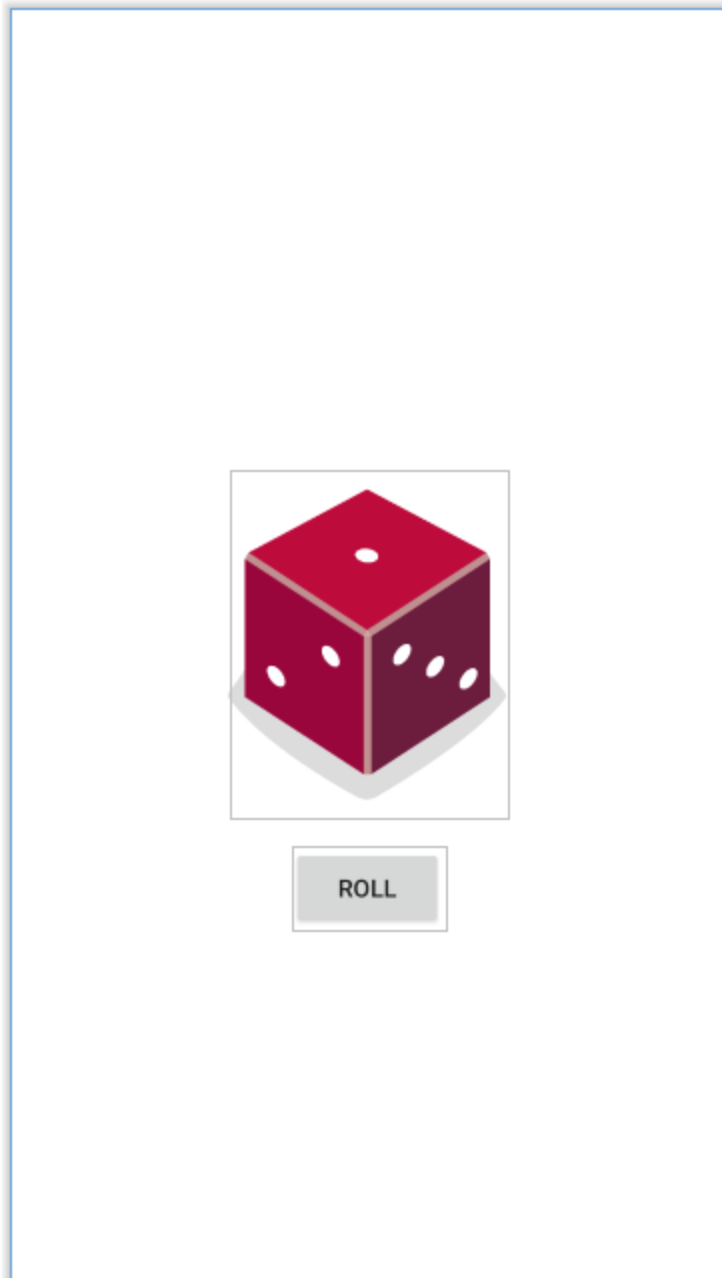
▼ **Layout**

Constraint Widget

▼ **Constraints**

- Start → StartOf **parent** (0dp)
- End → EndOf **parent** (0dp)
- Top → BottomOf **imageView** (16...
- Horizontal Bias (0.5)

Once the **Design** view updates, the app looks much better!

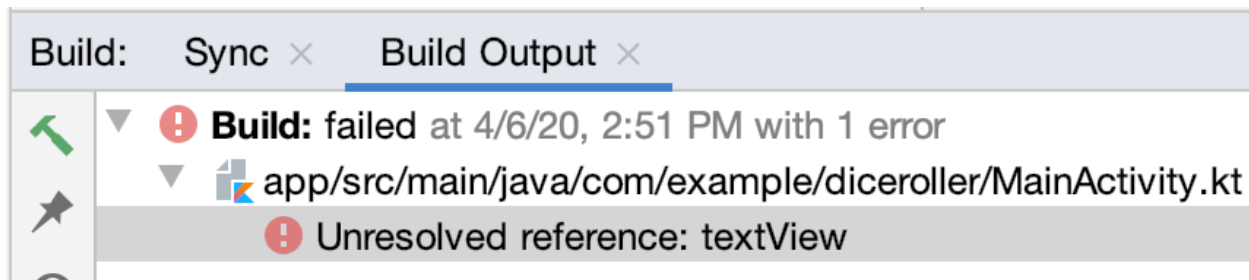


Note: Use density independent pixels (dp) as the unit to define these dimensions so that the image size scales appropriately on devices with different pixel resolution.

Change the dice image when the button is clicked

The layout has been fixed, but the `MainActivity` class needs to be updated to use the dice images.

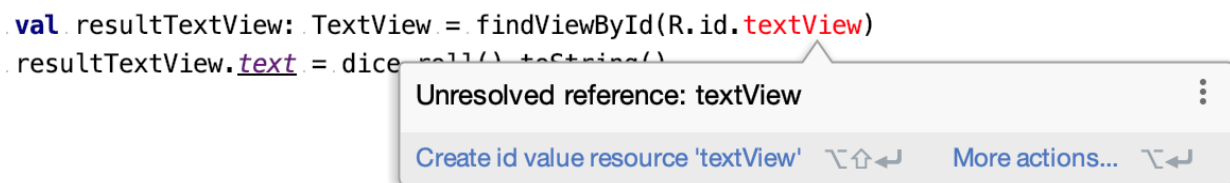
There is currently an error in the app in the `MainActivity.kt` file. If you try to run the app, you'll see this build error:



This is because your code is still referencing the `TextView` that you deleted from the layout.

1. Open `MainActivity.kt` (**app > java > com.example.diceroller > MainActivity.kt**)

The code refers to `R.id.textView`, but Android Studio doesn't recognize it.



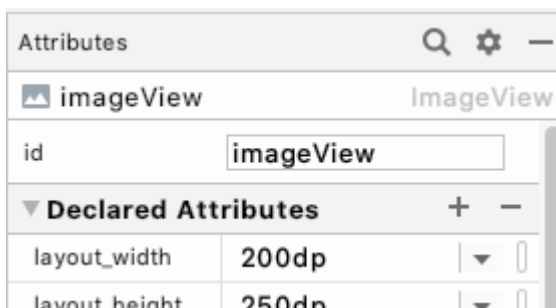
2. Within the `rollDice()` method, select any code that refers to `TextView` and delete it.

```
// Update the TextView with the dice roll
val resultTextView: TextView = findViewById(R.id.textView)
resultTextView.text = dice.roll().toString()
```

3. Still within `rollDice()`, create a new variable called `diceImage` of type `ImageView`. Set it equal to the `ImageView` from the layout. Use the `findViewById()` method and pass in the resource ID for the `ImageView`, `R.id.imageView`, as the input argument.

```
val diceImage: ImageView = findViewById(R.id.imageView)
```

If you're wondering how to figure out the precise resource ID of the `ImageView`, check the **id** at the top of the **Attributes** window.



When you refer to this resource ID in Kotlin code, make sure you type it exactly the same (lowercase i, capital V, no spaces). Otherwise Android Studio will show an error.

4. Add this line of code to test that you can correctly update the `ImageView` when the button is clicked. The dice roll will not always be "2" but just use the `dice_2` image for testing purposes.

```
diceImage.setImageResource(R.drawable.dice_2)
```

This code calls the `setImageResource()` method on the `ImageView`, passing the resource ID for the `dice_2` image. This will update the `ImageView` on screen to display the `dice_2` image.

The `rollDice()` method should look like this now:

```
private fun rollDice() {  
    val dice = Dice(6)  
    val diceRoll = dice.roll()  
    val diceImage: ImageView = findViewById(R.id.imageView)  
    diceImage.setImageResource(R.drawable.dice_2)  
}
```

5. Run your app to verify that it runs without errors. The app should start off with a blank screen except for the **Roll** button.

3:12



Dice Roller

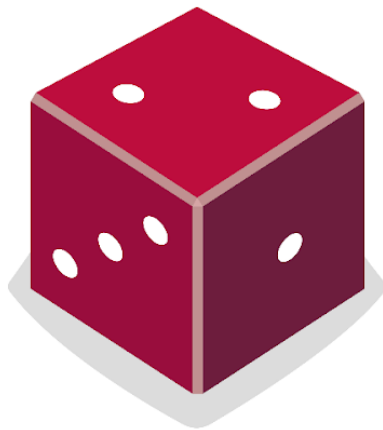
ROLL

Once you tap the button, a dice image displaying the value 2 will appear. Yes!!

12:50



Dice Roller



ROLL



You were able to change the image based on the button tap! You're getting closer!

5. Display the correct dice image based on the dice roll

Clearly the dice result won't always be a 2. Use the control flow logic that you learned in the [Add Conditional Behavior for Different Dice Rolls](#) codelab so that the appropriate dice image will be displayed on screen depending on the random dice roll.

Before you start to type code, think conceptually about how the app should behave by writing some *pseudocode* that describes what should happen. For example:

If the user rolls a 1, then display the `dice_1` image.

If the user rolls a 2, then display the `dice_2` image.

etc...

Note: *Pseudocode* is an informal description of how some code might work. It uses some elements of computer language like `if / else`, but describes things in a human understandable way. It can be useful for planning the correct approach to take before all the details have been decided.

The above pseudocode can be written with `if / else` statements in Kotlin based on the value of the dice roll.

```
if (diceRoll == 1) {  
    diceImage.setImageResource(R.drawable.dice_1)  
} else if (diceRoll == 2) {  
    diceImage.setImageResource(R.drawable.dice_2)  
}  
...
```

Writing `if / else` for each case gets pretty repetitive, though. The same logic can be expressed more simply with a `when` statement. This is more concise (less code)! Use this approach in your app.

```
when (diceRoll) {  
    1 -> diceImage.setImageResource(R.drawable.dice_1)  
    2 -> diceImage.setImageResource(R.drawable.dice_2)  
    ...  
}
```

Update the `rollDice()` method

1. In the `rollDice()` method, delete the line of code that sets the image resource ID to `dice_2` image every time.

```
diceImage.setImageResource(R.drawable.dice_2)
```

2. Replace it with a `when` statement that updates the `ImageView` based on the `diceRoll` value.

```
when (diceRoll) {  
    1 -> diceImage.setImageResource(R.drawable.dice_1)  
    2 -> diceImage.setImageResource(R.drawable.dice_2)  
    3 -> diceImage.setImageResource(R.drawable.dice_3)  
    4 -> diceImage.setImageResource(R.drawable.dice_4)  
    5 -> diceImage.setImageResource(R.drawable.dice_5)  
    6 -> diceImage.setImageResource(R.drawable.dice_6)  
}
```

The `rollDice()` method should look like this when you're done with the changes.

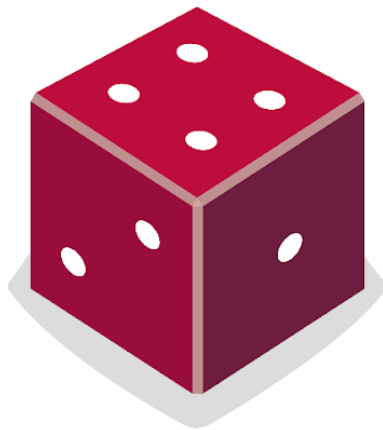
```
private fun rollDice() {  
    val dice = Dice(6)  
    val diceRoll = dice.roll()  
  
    val diceImage: ImageView = findViewById(R.id.imageView)  
  
    when (diceRoll) {  
        1 -> diceImage.setImageResource(R.drawable.dice_1)  
        2 -> diceImage.setImageResource(R.drawable.dice_2)  
        3 -> diceImage.setImageResource(R.drawable.dice_3)  
        4 -> diceImage.setImageResource(R.drawable.dice_4)  
        5 -> diceImage.setImageResource(R.drawable.dice_5)  
        6 -> diceImage.setImageResource(R.drawable.dice_6)  
    }  
}
```

3. Run the app. Clicking the **Roll** button changes the dice image to other values aside from 2. It works!

12:50



Dice Roller



ROLL



12:50



Dice Roller



ROLL



Optimize your code

If you want to write even more concise code, you could make the following code change. It doesn't have any visible impact to the user of your app, but it will make your code shorter and less repetitive.

You may have noticed that the call to `diceImage.setImageResource()` appears 6 times in your `when` statement.

```
when (diceRoll) {
    1 -> diceImage.setImageResource(R.drawable.dice_1)
    2 -> diceImage.setImageResource(R.drawable.dice_2)
    3 -> diceImage.setImageResource(R.drawable.dice_3)
    4 -> diceImage.setImageResource(R.drawable.dice_4)
    5 -> diceImage.setImageResource(R.drawable.dice_5)
    6 -> diceImage.setImageResource(R.drawable.dice_6)
}
```

The only thing that changes between each case is the resource ID that's being used. That means you can create a variable to store the resource ID to use. Then you can call `diceImage.setImageResource()` only once in your code and pass in the correct resource ID.

1. Replace the code above with the following.

```
val drawableResource = when (diceRoll) {
    1 -> R.drawable.dice_1
    2 -> R.drawable.dice_2
    3 -> R.drawable.dice_3
    4 -> R.drawable.dice_4
    5 -> R.drawable.dice_5
    6 -> R.drawable.dice_6
}

diceImage.setImageResource(drawableResource)
```

A new concept here is that a `when` expression can actually return a value. With this new code snippet, the `when` expression returns the correct resource ID, which will be stored in the `drawableResource` variable. Then you can use that variable to update the image resource displayed.

2. Notice that `when` is now underlined in red. If you hover your pointer over it, you'll see an error message: **'when' expression must be exhaustive, add necessary 'else' branch**.

The error is because the value of the `when` expression is assigned to `drawableResource`, so the `when` must be exhaustive—it must handle all the cases possible so that a value is always returned, even if you change to a 12-sided dice. Android Studio suggests adding an `else` branch. You can fix this by changing the case for 6 to `else`. The cases for 1 through 5 are the same, but all others including 6 are handled by the `else`.

```

val drawableResource = when (diceRoll) {
    1 -> R.drawable.dice_1
    2 -> R.drawable.dice_2
    3 -> R.drawable.dice_3
    4 -> R.drawable.dice_4
    5 -> R.drawable.dice_5
    else -> R.drawable.dice_6
}

diceImage.setImageResource(drawableResource)

```

3. Run the app to make sure it still works correctly. Be sure to test it enough to make sure that you see all the numbers appear with the dice images 1 through 6.

Set an appropriate content description on the ImageView

Now that you've replaced the rolled number with an image, screen readers cannot tell anymore what number was rolled. To fix this, after you've updated the image resource, update the content description of the `ImageView`. The content description should be a text description of what is shown in the `ImageView` so that screen readers can describe it.

```

diceImage.contentDescription = diceRoll.toString()

```

Screen readers can read aloud this content description, so if the dice roll of "6" image is displayed on the screen, the content description would be read out loud as "6".

Note: Normally, a content description should use string resources that can be translated into other languages, but we'll get into this in a future lesson.

6. Adopt good coding practices

Create a more useful launch experience

When the user opens the app for the first time, the app is blank (except the **Roll** button), which looks odd. Users may not know what to expect, so change the UI to display a random dice roll when you first start the app and create the `Activity`. Then users are more likely to understand that tapping the **Roll** button will produce a dice roll.

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    val rollButton: Button = findViewById(R.id.button)
    rollButton.setOnClickListener { rollDice() }

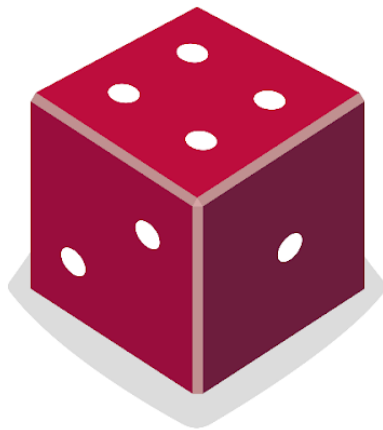
    // Do a dice roll when the app starts
    rollDice()
}

```

12:50



Dice Roller



ROLL



Comment Your Code

Add some comments to your code to describe what is happening in the code you wrote.

After you've made all these changes, this is what your `rollDice()` method might look like.

```
/**
 * Roll the dice and update the screen with the result.
 */
private fun rollDice() {
    // Create new Dice object with 6 sides and roll the dice
    val dice = Dice(6)
    val diceRoll = dice.roll()

    // Find the ImageView in the layout
    val diceImage: ImageView = findViewById(R.id.imageView)

    // Determine which drawable resource ID to use based on the dice roll
    val drawableResource = when (diceRoll) {
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }

    // Update the ImageView with the correct drawable resource ID
    diceImage.setImageResource(drawableResource)

    // Update the content description
    diceImage.contentDescription = diceRoll.toString()
}
```

For the full `MainActivity.kt` file, see the solution code on GitHub linked in the next step.

Well done on completing the Dice Roller app! Now you can bring this app to the next game night with your friends!

7. Solution code

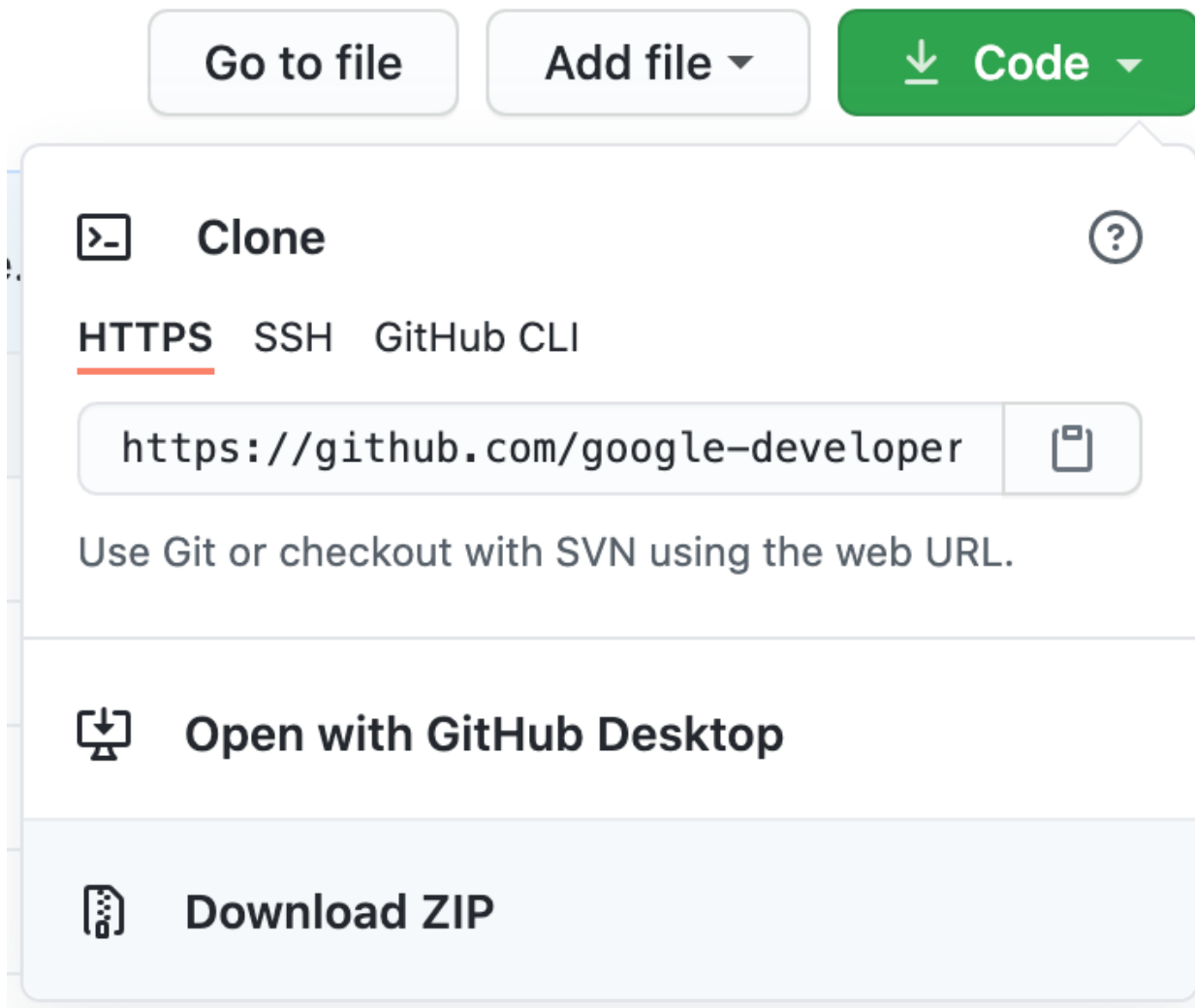
The solution code for this codelab is in the project and module shown below.

Solution Code URL: <https://github.com/google-developer-training/android-basics-kotlin-dice-roller-with-images-app-solution>

To get the code for this codelab and open it in Android Studio, do the following.

Get the code

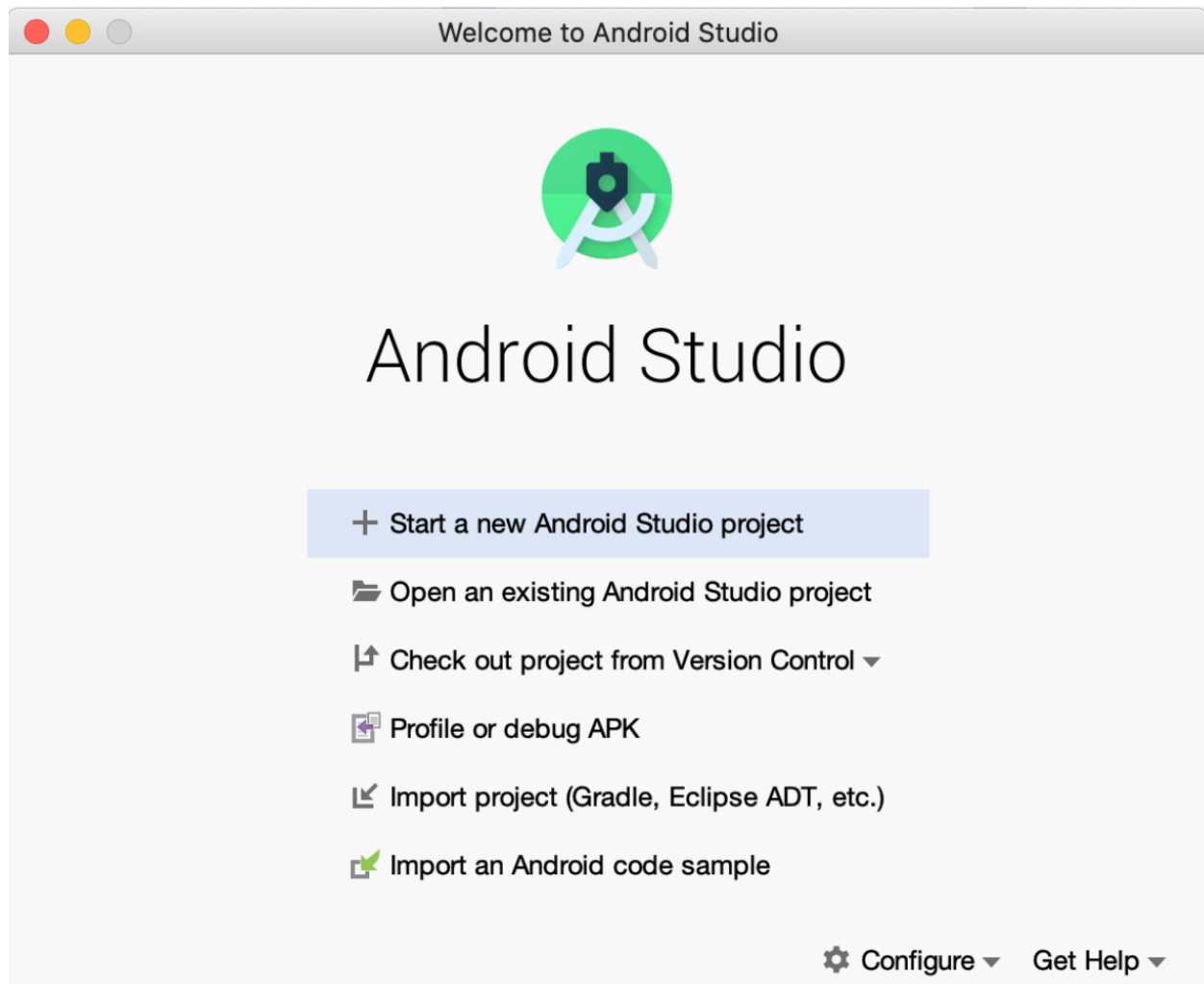
1. Click on the provided URL. This opens the GitHub page for the project in a browser.
2. On the GitHub page for the project, click the **Code** button, which brings up a dialog.



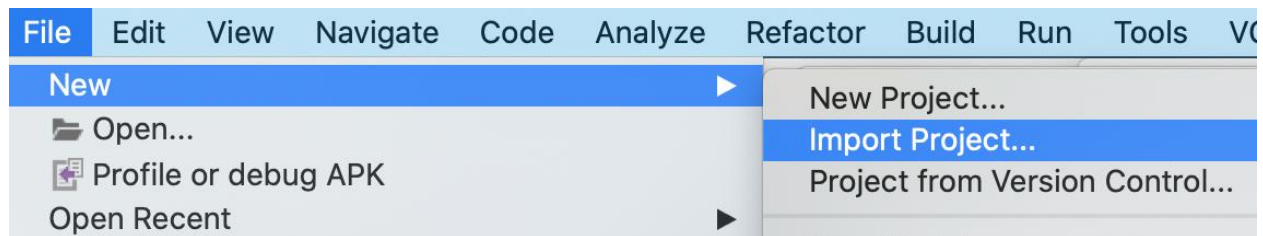
3. In the dialog, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
4. Locate the file on your computer (likely in the **Downloads** folder).
5. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

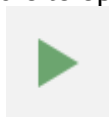
1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open an existing Android Studio project**.



Note: If Android Studio is already open, instead, select the **File > New > Import Project** menu option.



3. In the **Import Project** dialog, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.



6. Click the **Run** button to build and run the app. Make sure it builds as expected.

7. Browse the project files in the **Project** tool window to see how the app is set-up.

8. Summary

- Use [`setImageResource\(\)`](#) to change the image that's displayed in an `ImageView`
- Use control flow statements like `if / else` expressions or `when` expressions to handle different cases in your app, for example, showing different images under different circumstances.

9. Learn more

- [Android Kotlin glossary](#)
- [if expression in Kotlin](#)
- [when expression in Kotlin](#)
- [ImageView.setImageResource\(\)](#)
- [Accessibility](#)

10. Practice on your own

Note: Practices are optional. They provide an opportunity for you to practice what you've learned in this codelab.

Do the following:

1. Add another dice to the app, so that one **Roll** button gives 2 dice results. How many `ImageViews` will you need in your layout? How will that affect the `MainActivity.kt` code?

Check your work:

Your finished app should run without errors and show the two dice.